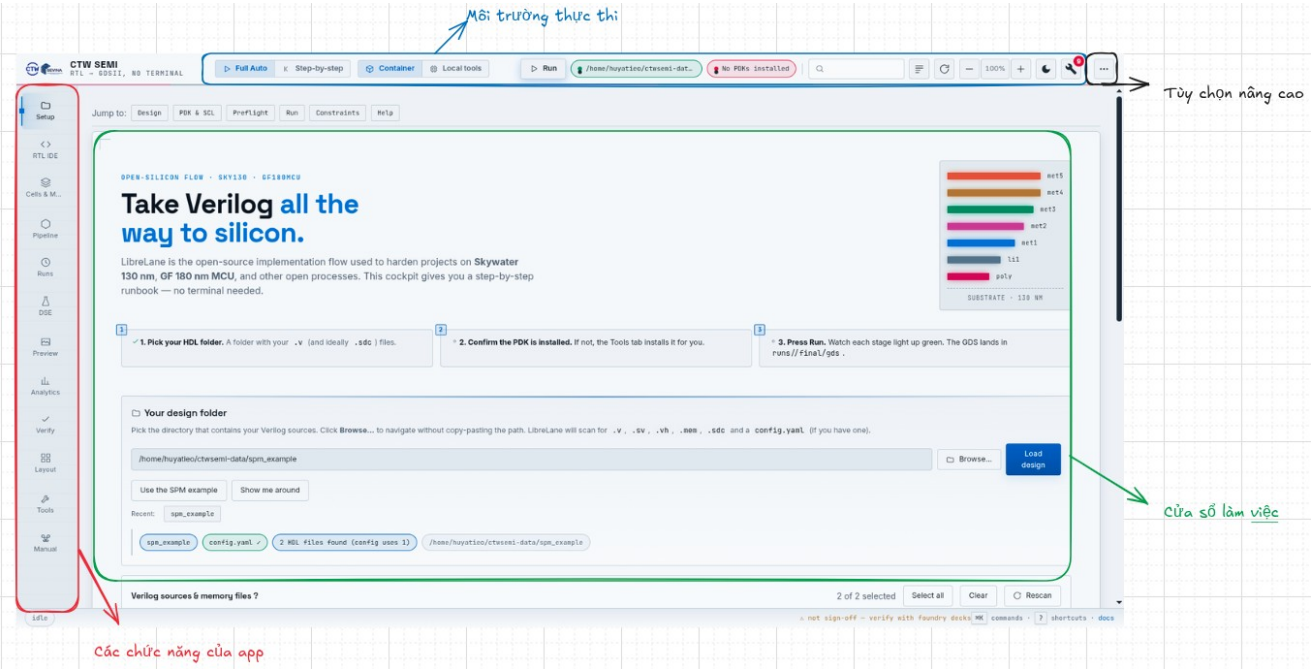
LAB 0: SỬA FLOW FlOOR PLANNING

1 Làm quen tools

1.1 Giao diện app

Mục tiêu ở phần này là nắm được phần hiển thị của app.



Giao diện app gồm 3 khu vực như được tô màu ở trên gồm:

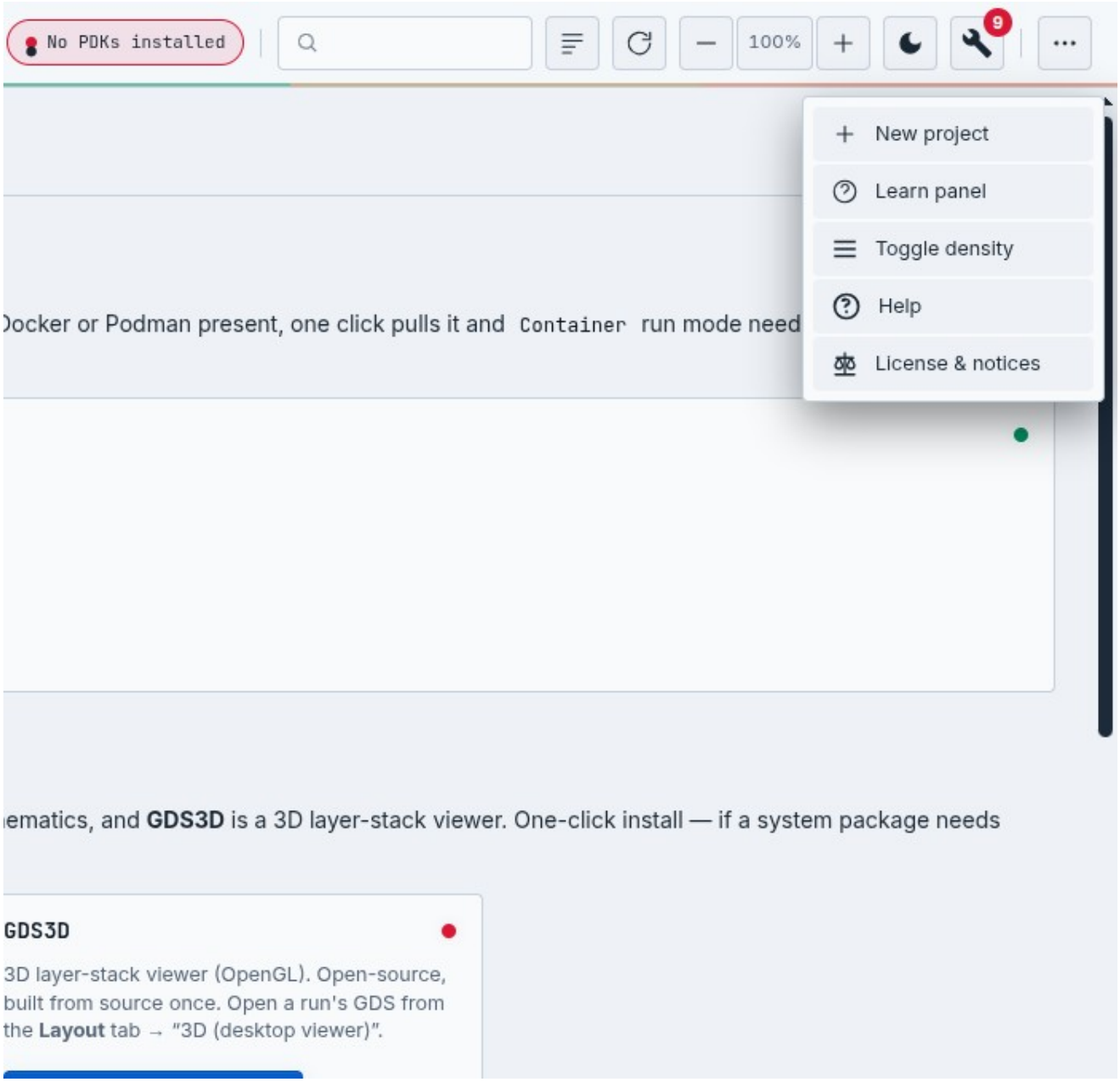
- **Các chức năng của app**:

- Setup: chọn design, PDK, SCL, constraints và preflight.
- `RTL IDE`: sửa RTL, lint, mô phỏng, xem waveform.
- `Pipeline`: xem các bước RTL → GDS và log trực tiếp.
- `Runs`: lịch sử run, trạng thái pass/fail, tải bundle.
- `Preview`: xem hình layout, GDS, DEF, netlist và file đầu ra.
- `Analytics`: xem metric của từng công đoạn.
- `Verify`: DRC, LVS, timing, antenna và XOR.
- `DSE`: chạy nhiều cấu hình để so sánh.
- `Layout`: mở bằng KLayout, Magic hoặc GDS3D.
- `Tools`: quản lý Docker/Podman, LibreLane image và PDK.
- `Manual`: xem và chạy lệnh LibreLane tương ứng.
- `Cells & Macros`: standard cell, custom cell và hard macro.

Trong lab 0 này, ta chỉ sử dụng: `Tools` → `Setup` → `Pipeline` → `Runs` → `Preview` → `Verify`.

- **Môi trường thực thi**: Lựa chọn chế độ hoạt động (full auto, manual), thư mục làm việc, chuyển đổi hiển thị sáng tối, trạng thái hoạt động của tools,…

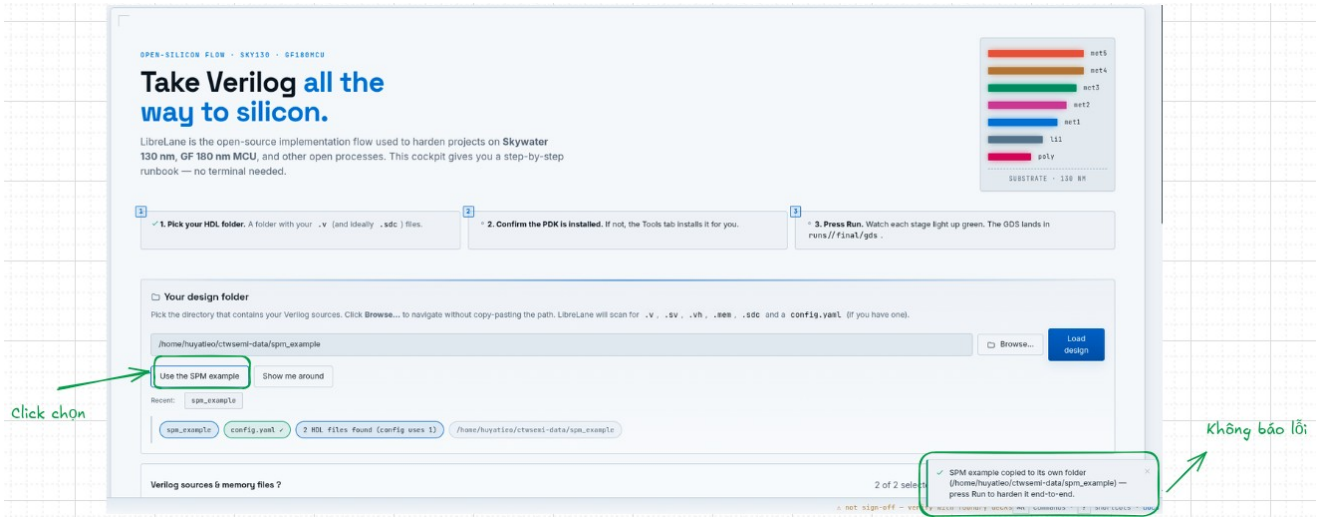
- **Tùy chọn nâng cao**: New project để tạo dự án mới, Learn panel để học các bước trong RTL to GDS flow, Toggle Density để chọn kiểu hiển thị, Help để hướng dẫn nhanh các lệnh.



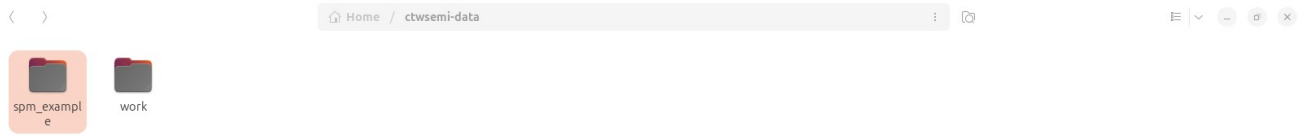
1.2 Chạy dự án mẫu

Mục tiêu phần này là chạy dự án mẫu có sẵn, và chạy 1 mạch auto ra GDS. Để xem có cài đủ công cụ hay chưa và có báo lỗi trong quá trình chạy hay không.

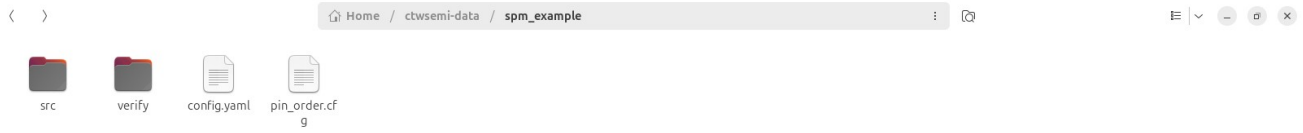
Ở thanh chức năng của app. Chọn **Setup,** tại cửa sổ làm việc chọn **use the SPM example,** để thêm vào thư mục đang chứa ví dụ về bộ nhân nối tiếp song song (SPM).



Nếu báo lỗi, kiểm tra thư mục có tên **ctwsemi-data**, bên trong xem thử có spm_example không, nếu không có thì chạy lại lệnh tạo docker lần đầu để tải lại. Mặc định là đã có thư mục này trong lúc tải docker rồi:



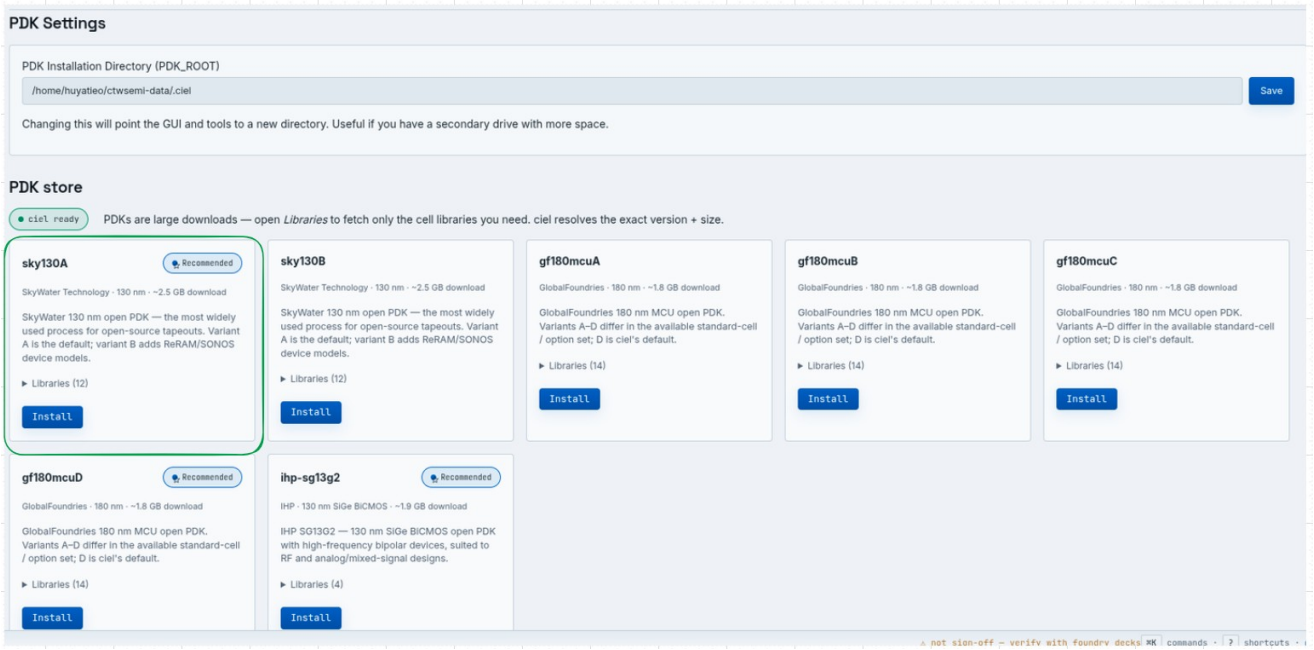
Nhìn vào bên trong spm_example:



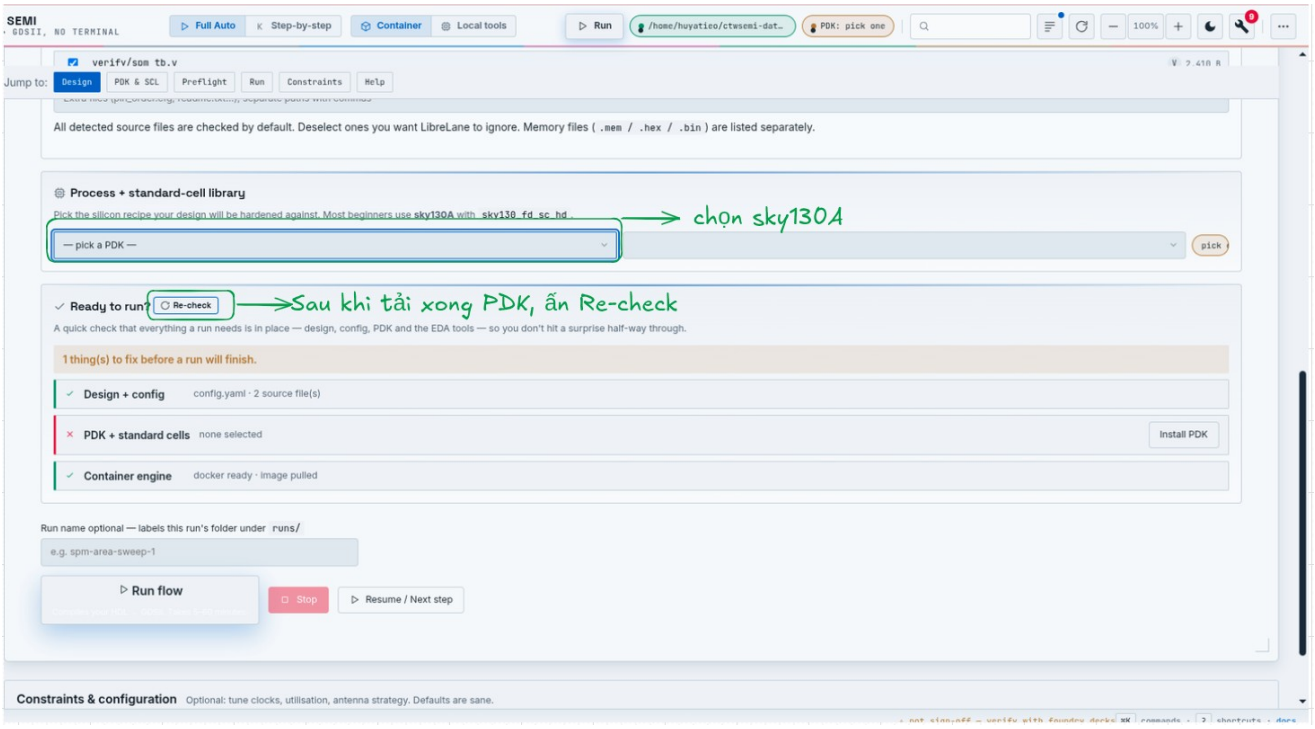
- src: nơi chứa code RTL viết bằng verilog, kết thúc là file .v, file constraint về timing, kết thúc bằng file .sdc
- verify: chứa file testbench để kiểm thử chức năng của SPM. tb_spm.
- config.yaml: nơi cấu hình cho flow chạy: tên top module (nếu có nhiều file thiết kế), tên của tín hiệu clock, chu kì clock, tên constraint, diện tích chip, pdk, phân bổ mật độ.
- pin_order.yaml: cấu hình chân và thứ tự.

Tiếp theo, ta cuộn xuống dưới, sẽ thấy mục **Run flow,** trên thanh thông báo phần PDK sẽ hiện dấu ‘X’ vì chưa tải. Chọn Install PDK, sẽ điều hướng sang cửa sổ PDK , chọn sky130A trong mục PDK store, phần này cần 3gb trống, và sẽ tải tầm khoảng 3-5 phút.

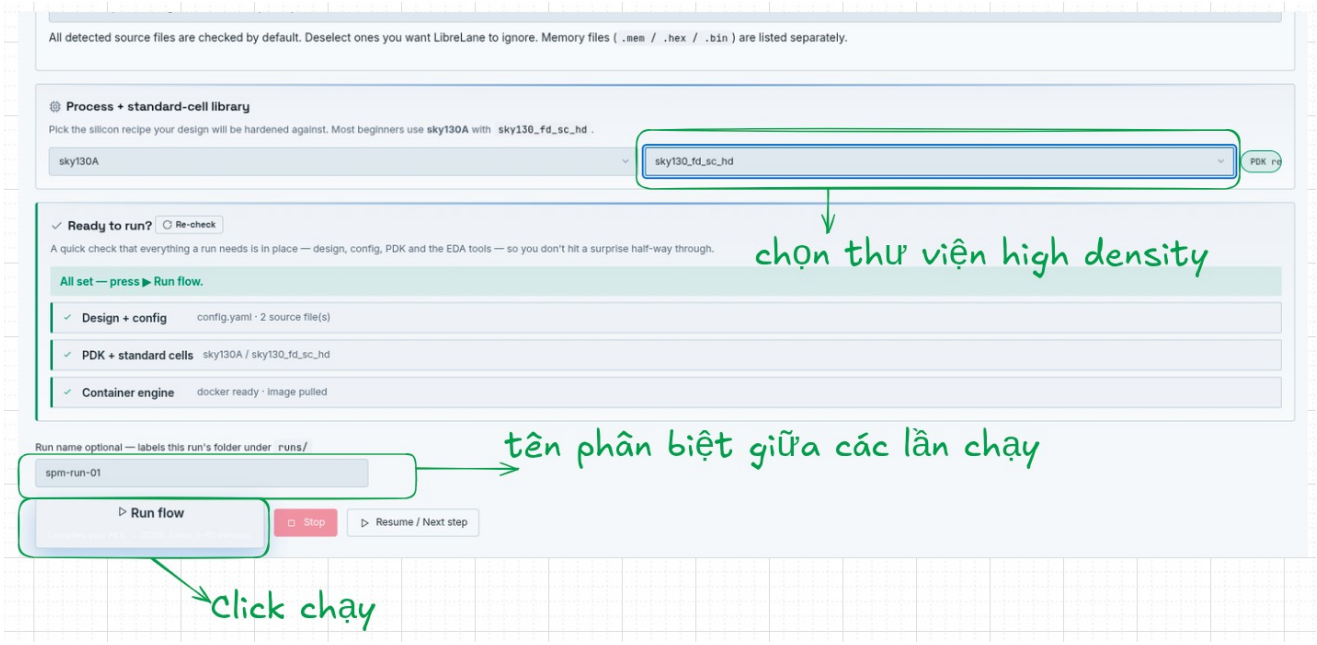
PDK store:



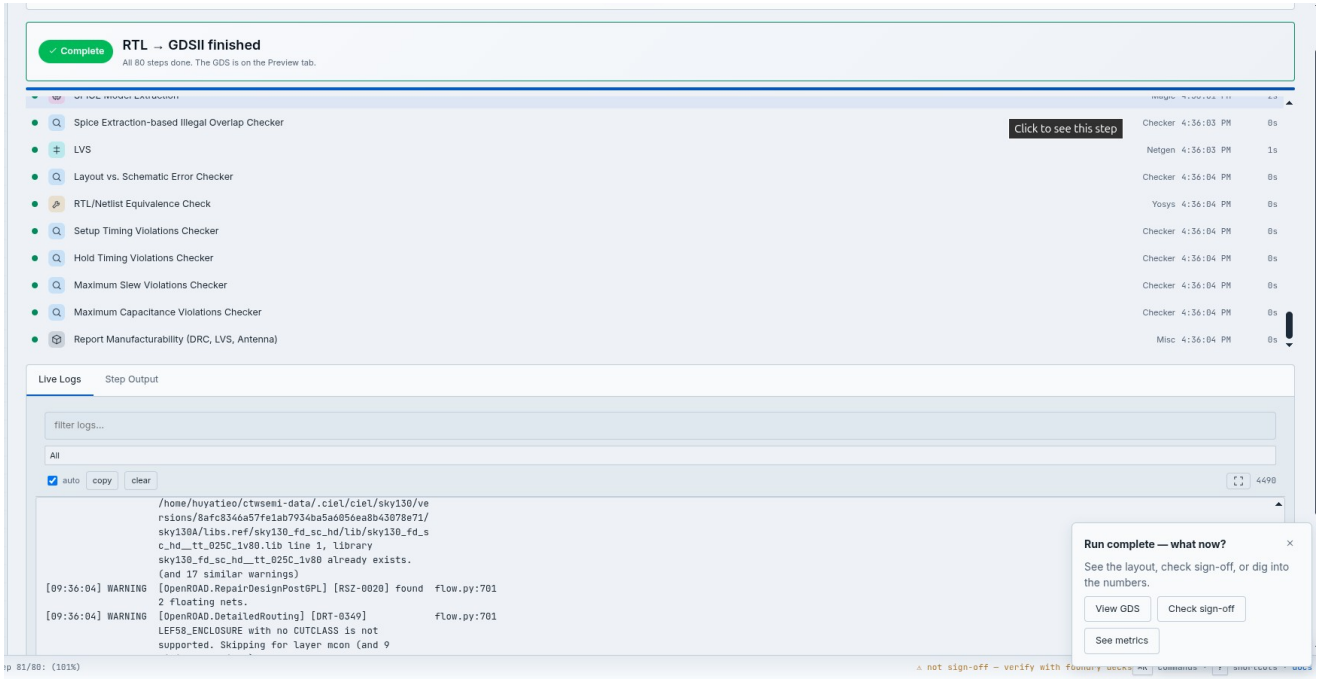
Sau khi tải thành công PDK, bấm chọn như hình dưới.



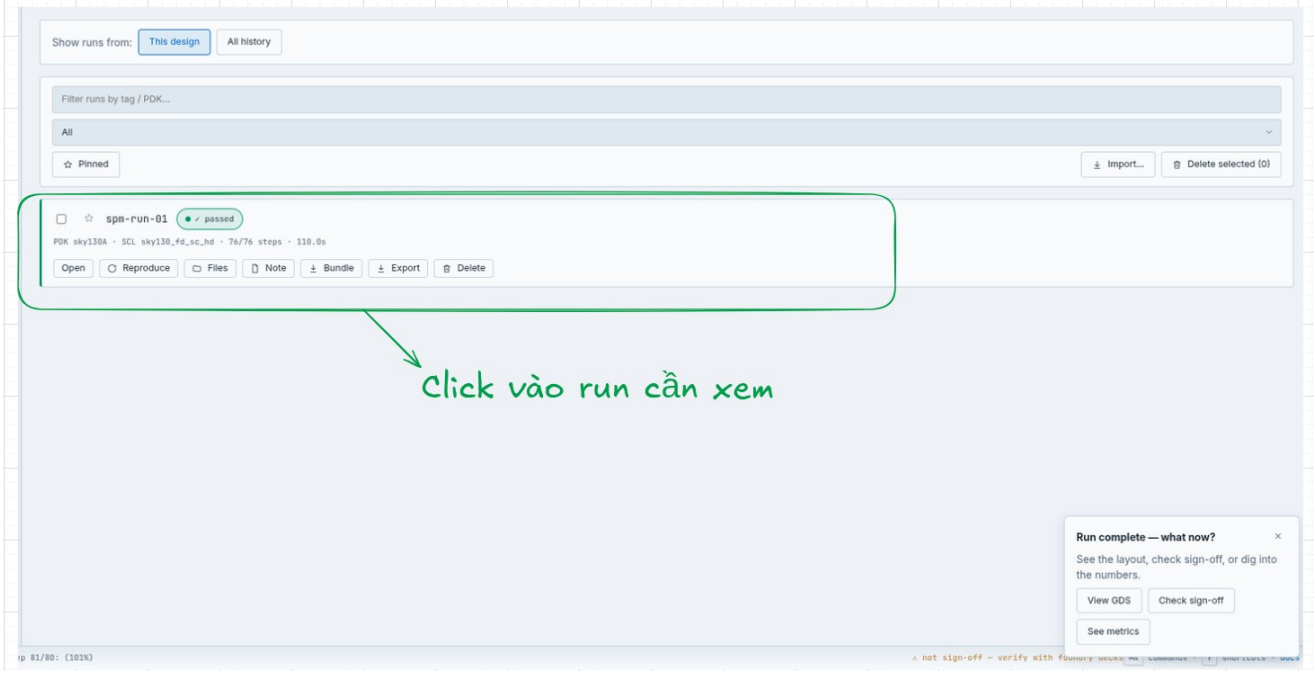
Kết quả phải như hình dưới:



Chờ tool chạy và hiển thị như dưới đây và không báo lỗi là ok.



Trên thanh chức năng, chọn **Runs**, vào cửa sổ, chọn spm-run-01.



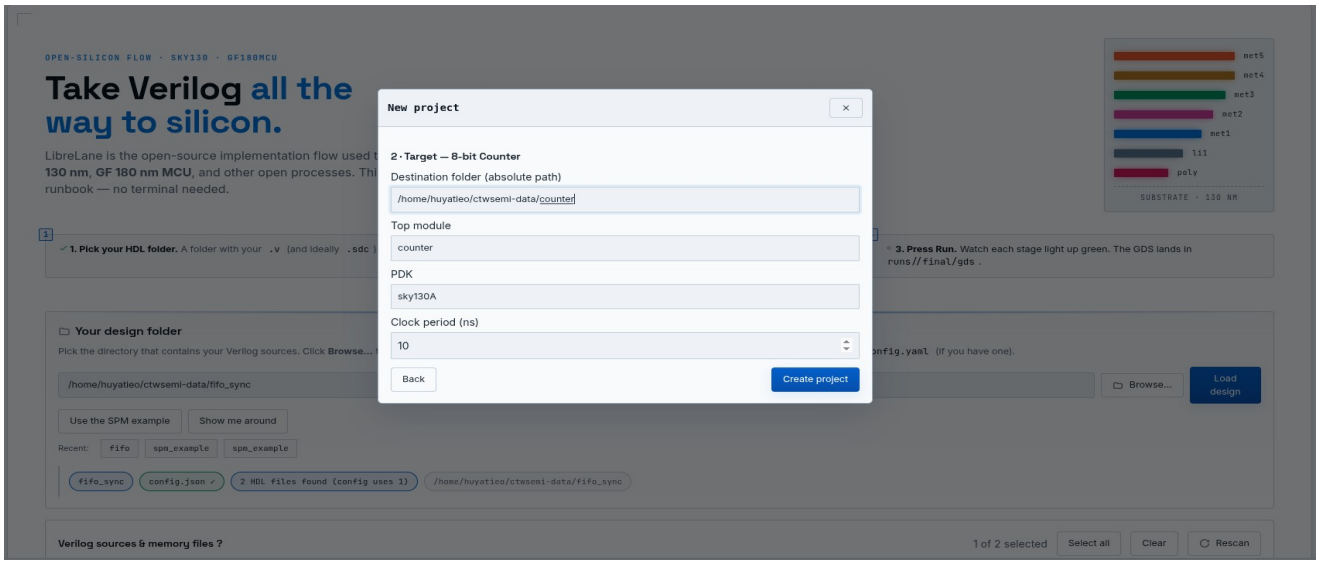
Cửa sổ GDSII hiện ra



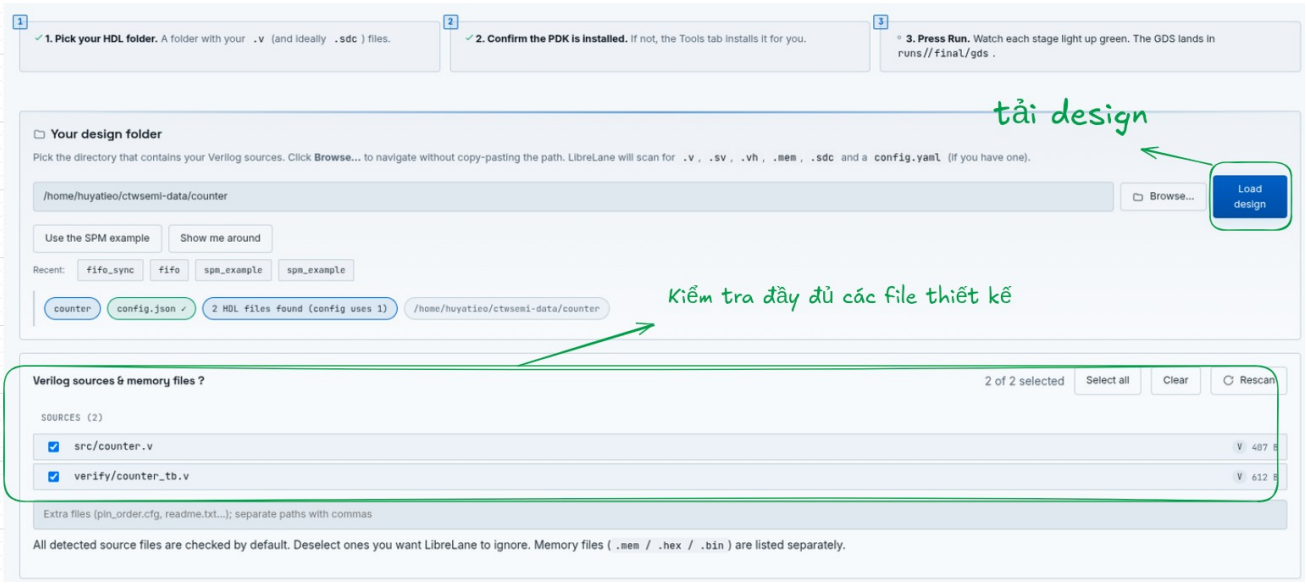
1.3 Thiết lập dự án counter 8

Chọn tùy chọn nâng cao, biểu tượng dấu 3 chấm (xem giao diện app ở phần 1.1). Chọn ví dụ **8-bit counter**. Chọn vị trí để lưu, ví dụ /home/huyatieo/ctwsemi-data/counter, học viên có thể chọn đường dẫn bất kỳ bên ngoài ctwsemi-data. Chọn clock mặc định là 10ns (100MHz) và PDK sky130A.

Bấm **Create project** sau khi thiết lập giống như hình bên dưới (khác đường dẫn):



Lúc này, ở phần Setup, sẽ xuất hiện đường dẫn của dự án, chọn **Load design** để kiểm tra các file RTL(.v,.sv,.sh), file ràng buộc (.sdc) và file cấu hình (config.yaml) có trong dự án.



1.4 Lint check và kiểm thử chức năng

Bước này ta tiến hành kiểm tra cú pháp của code RTL và chức năng của RTL. Nếu bước này PASS và chức năng đúng như ta mong đợi, thì mới chạy các bước kế tiếp.

Mở 1 terminal (cửa sổ dòng lệnh) mới trong Ubuntu (WSL hoặc Vmware) chạy lệnh dưới để cài Iverilog. (chỉ cài đúng 1 lần, những lần sau không cần cài).

```
sudo docker exec -u 0 ctwsemi sh -lc '
apt-get update &&
apt-get install -y --no-install-recommends iverilog &&
rm -rf /var/lib/apt/lists/*
'

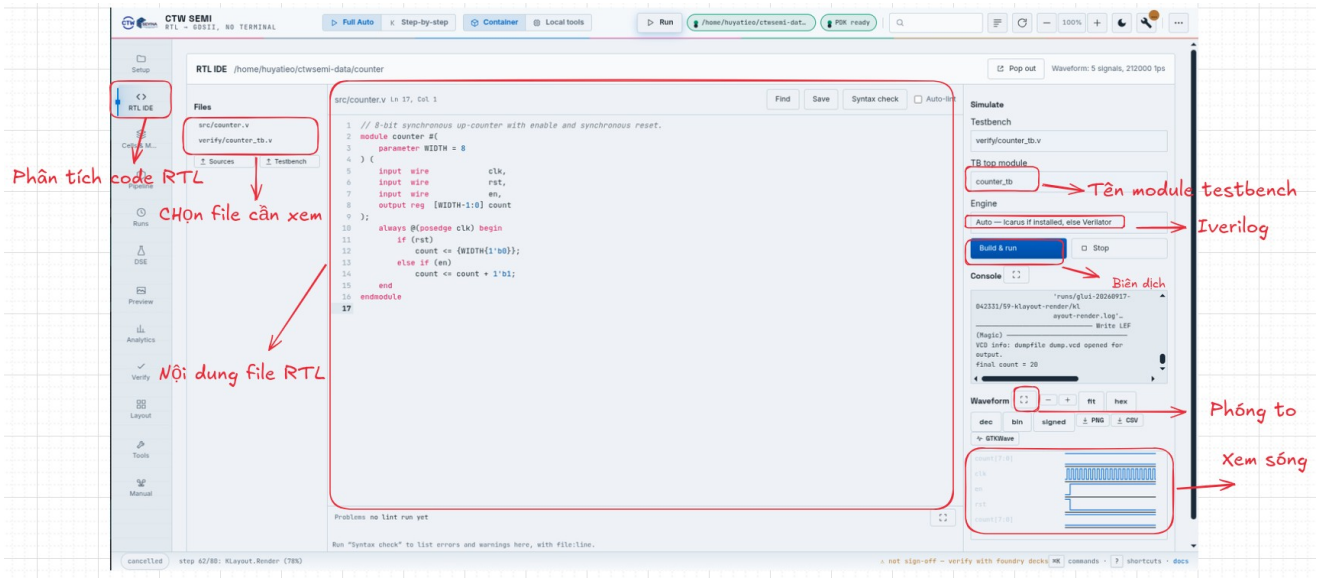
sudo docker exec ctwsemi sh -lc '
iverilog -V | head -n 1
command -v vvp
'

sudo docker restart ctwsemi
```

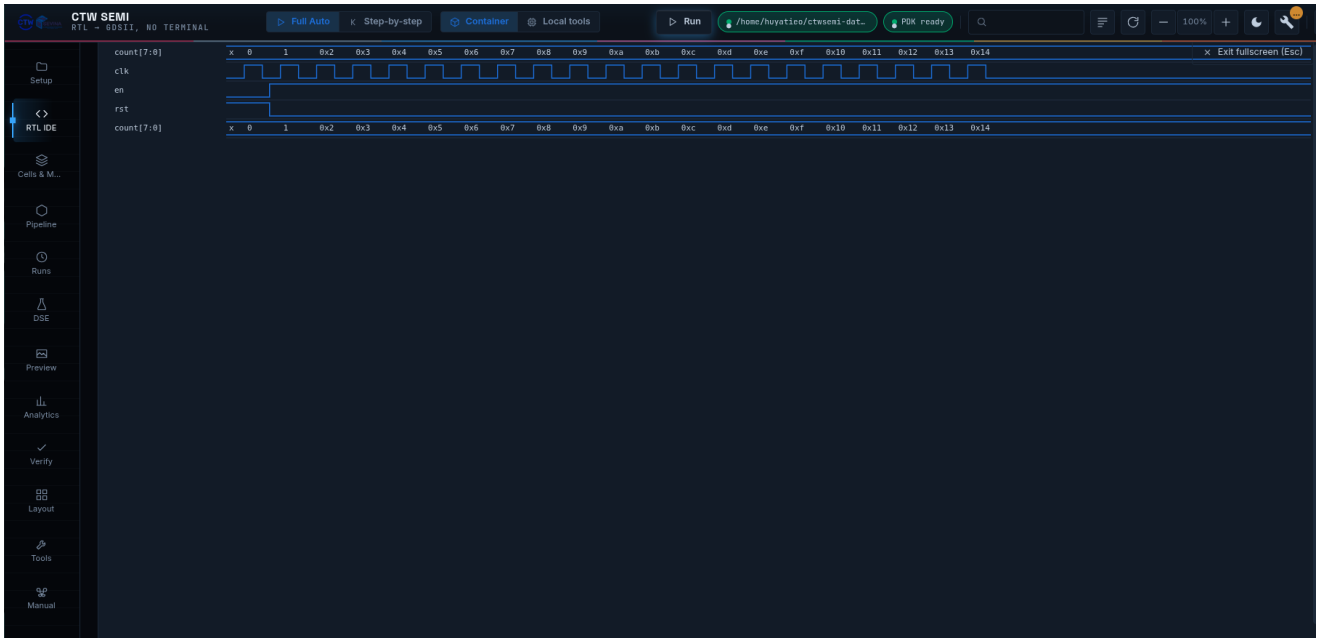
Sau khi tải xong, vào lại app, chọn nút Reload CTW SEMI hoặc tổ hợp Ctrl+R.

Cửa sổ RTL IDE:

Chọn RTL IDE ở thanh chức năng, kiểm tra nội dung của 2 file, trong đó file counter.v chứa module counter là thiết kế RTL của chúng ta. Còn file counter_tb.v là file kiểm thử chức năng của counter.



Kiểm tra chức năng đã đúng chưa bằng cách phóng to cửa sổ sóng Waveform, chỉnh chế độ sang xem tối (biểu tượng hình mặt trăng trên thanh trạng thái):



1.5 Run flow với auto mode

Mục tiêu của bước này là chạy flow GDSII lần đầu để kiểm tra các cấu hình mặc định có hoàn thành được tới GDSII không. Tìm file config.json tại /home/huyatieo/ctwsemi-data/ (chọn đúng đường dẫn của học viên):

Open

config.json
~/ctwsemi-data/counter

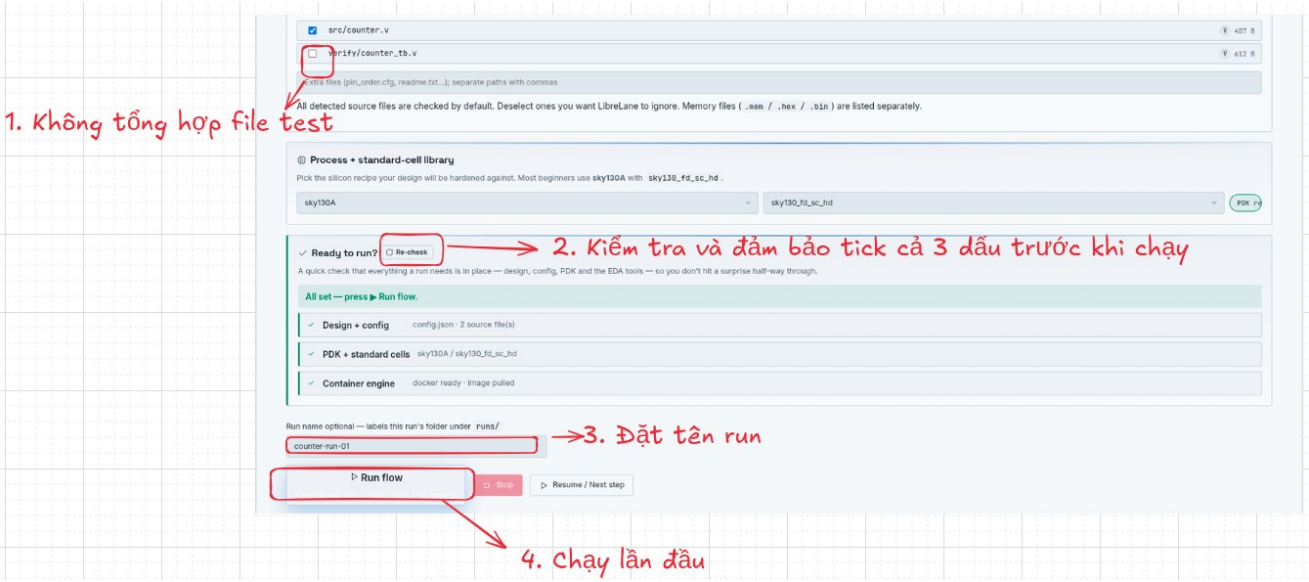
```
{
  "DESIGN_NAME": "counter",
  "VERILOG_FILES": [
    "dir::src/*.v"
  ],
  "CLOCK_PORT": "clk",
  "CLOCK_PERIOD": 10.0,
  "PDK": "sky130A"
}
```

Giải thích Cấu hình cho counter hiện tại như sau:

- “DESIGN_NAME”: “counter” tên của module cần chạy gds, module này gọi là module top.
- “VERILOG_FILES” nơi chứa file RTL (file có đuôi .v).
- “CLOCK_PORT”: tên của chân tín hiệu clock, với ví dụ này là “clk”.
- “CLOCK_PERIOD”: chu kì của clock, đơn vị ns, với ví dụ này là 10ns (100MHz).
-“PDK”: thư viện công nghệ dùng, với ví dụ này là sky130A.

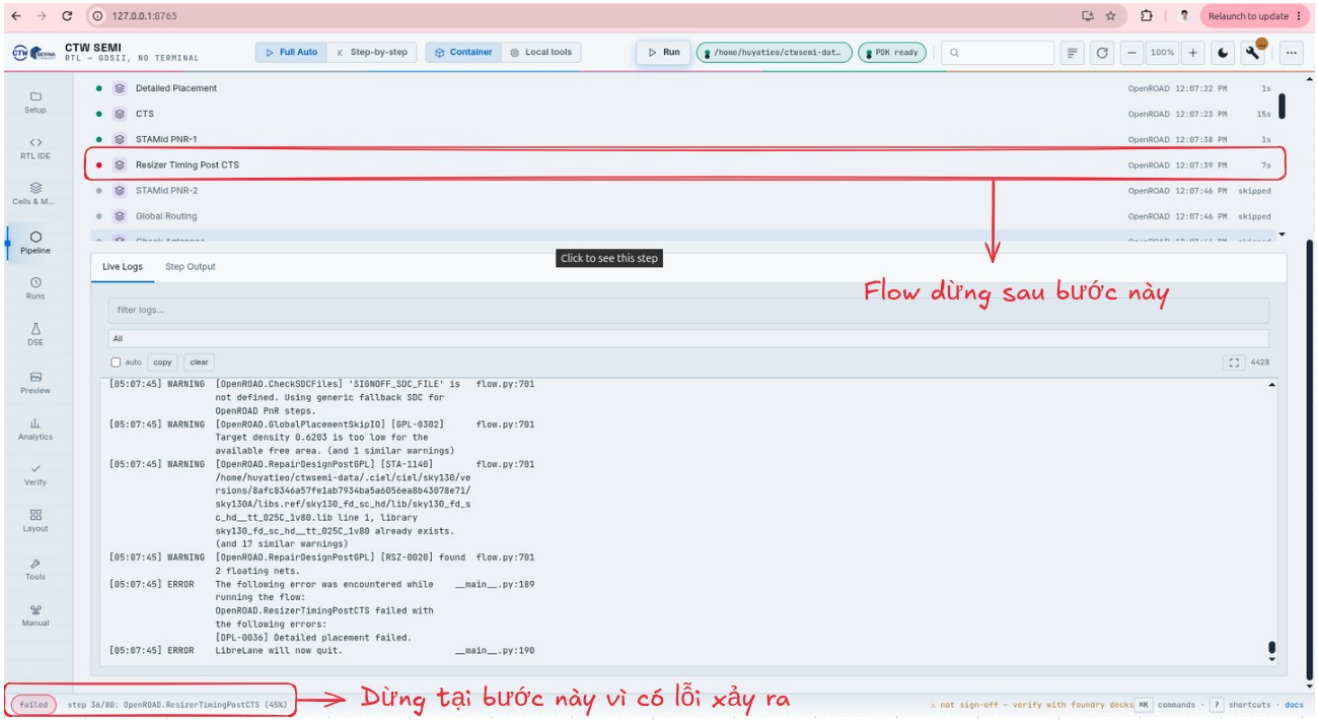
Các cấu hình khác như diện tích của die chip, diện tích core, mật độ phân bổ, ta để mặc định.

Làm các bước theo thứ tự đánh số như hình dưới.



1.6 Sửa cấu hình và phân tích

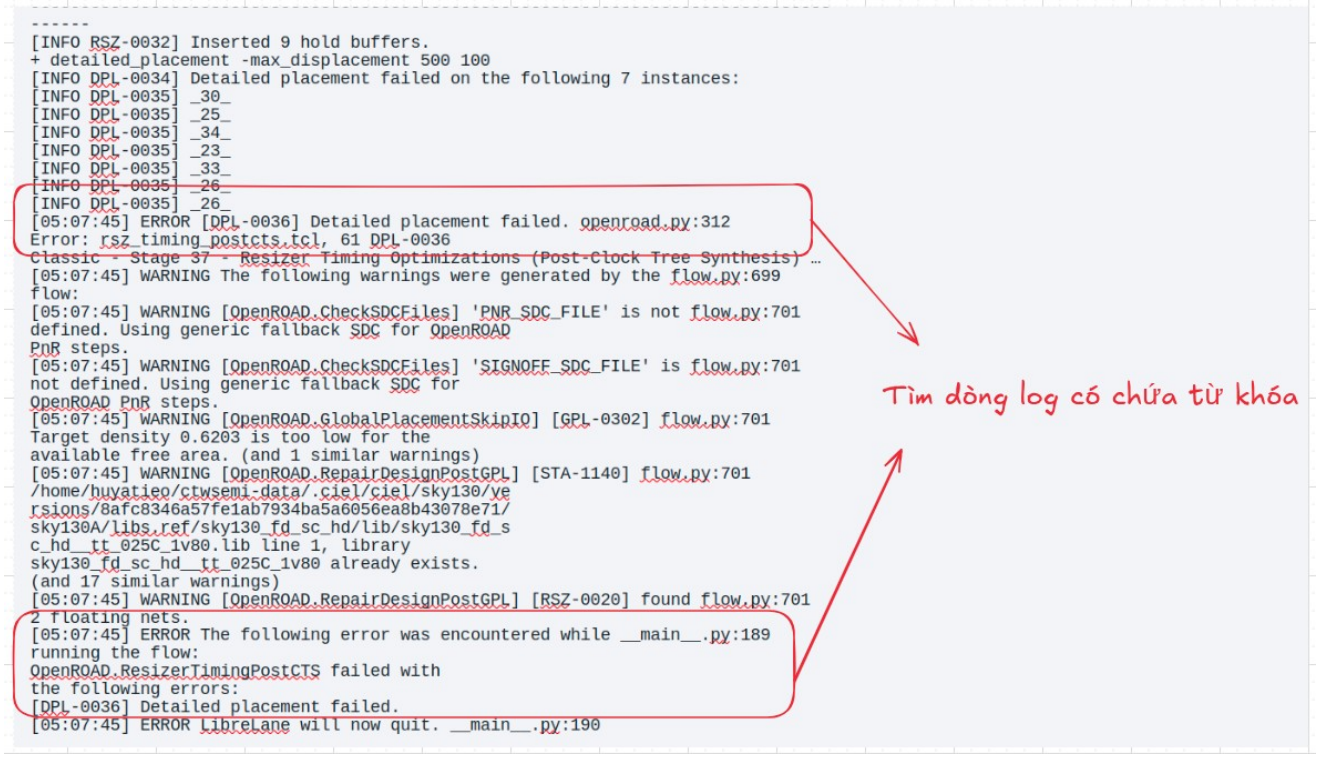
Ngay khi chạy lần đầu, hệ thống báo lỗi và dừng lại ngay tại Resizer Timing Post CTS.



Chi tiết hơn về lỗi, ta copy output trả về:

```
[INFO RSZ-0098] No setup violations found
+ repair_timing -verbose -hold -setup_margin
0.05000000000000000277555756156289135105907917022705078125 -hold_margin
0.10000000000000000055511151231257827021181583404541015625 -max_buffer_percent
50.0
[INFO RSZ-0046] Found 7 endpoints with hold violations.
Iteration | Resized | Buffers | Cloned Gates | Area | WNS | TNS |
Endpoint
---------------------------------------------------------------------------------
------
0 | 0 | 0 | 0 | +0.0% | 0.024 | 0.000 |
_48_/D
final | 0 | 9 | 0 | +15.8% | 0.102 | 0.000 |
_46_/D
---------------------------------------------------------------------------------
------
[INFO RSZ-0032] Inserted 9 hold buffers.
+ detailed_placement -max_displacement 500 100
[INFO DPL-0034] Detailed placement failed on the following 7 instances:
[INFO DPL-0035] _30_
[INFO DPL-0035] _25_
[INFO DPL-0035] _34_
[INFO DPL-0035] _23_
[INFO DPL-0035] _33_
[INFO DPL-0035] _26_
[INFO DPL-0035] _26_
[05:07:45] ERROR [DPL-0036] Detailed placement failed. openroad.py:312
Error: rsz_timing_postcts.tcl, 61 DPL-0036
Classic - Stage 37 - Resizer Timing Optimizations (Post-Clock Tree Synthesis) …
[05:07:45] WARNING The following warnings were generated by the flow.py:699
flow:
[05:07:45] WARNING [OpenROAD.CheckSDCFiles] 'PNR_SDC_FILE' is not flow.py:701
defined. Using generic fallback SDC for OpenROAD
PnR steps.
[05:07:45] WARNING [OpenROAD.CheckSDCFiles] 'SIGNOFF_SDC_FILE' is flow.py:701
not defined. Using generic fallback SDC for
OpenROAD PnR steps.
[05:07:45] WARNING [OpenROAD.GlobalPlacementSkipIO] [GPL-0302] flow.py:701
Target density 0.6203 is too low for the
available free area. (and 1 similar warnings)
[05:07:45] WARNING [OpenROAD.RepairDesignPostGPL] [STA-1140] flow.py:701
/home/huyatieo/ctwsemi-data/.ciel/ciel/sky130/ve
rsions/8afc8346a57fe1ab7934ba5a6056ea8b43078e71/
sky130A/libs.ref/sky130_fd_sc_hd/lib/sky130_fd_s
c_hd__tt_025C_1v80.lib line 1, library
sky130_fd_sc_hd__tt_025C_1v80 already exists.
(and 17 similar warnings)
[05:07:45] WARNING [OpenROAD.RepairDesignPostGPL] [RSZ-0020] found flow.py:701
2 floating nets.
[05:07:45] ERROR The following error was encountered while __main__.py:189
running the flow:
OpenROAD.ResizerTimingPostCTS failed with
the following errors:
[DPL-0036] Detailed placement failed.
[05:07:45] ERROR LibreLane will now quit. __main__.py:190
```

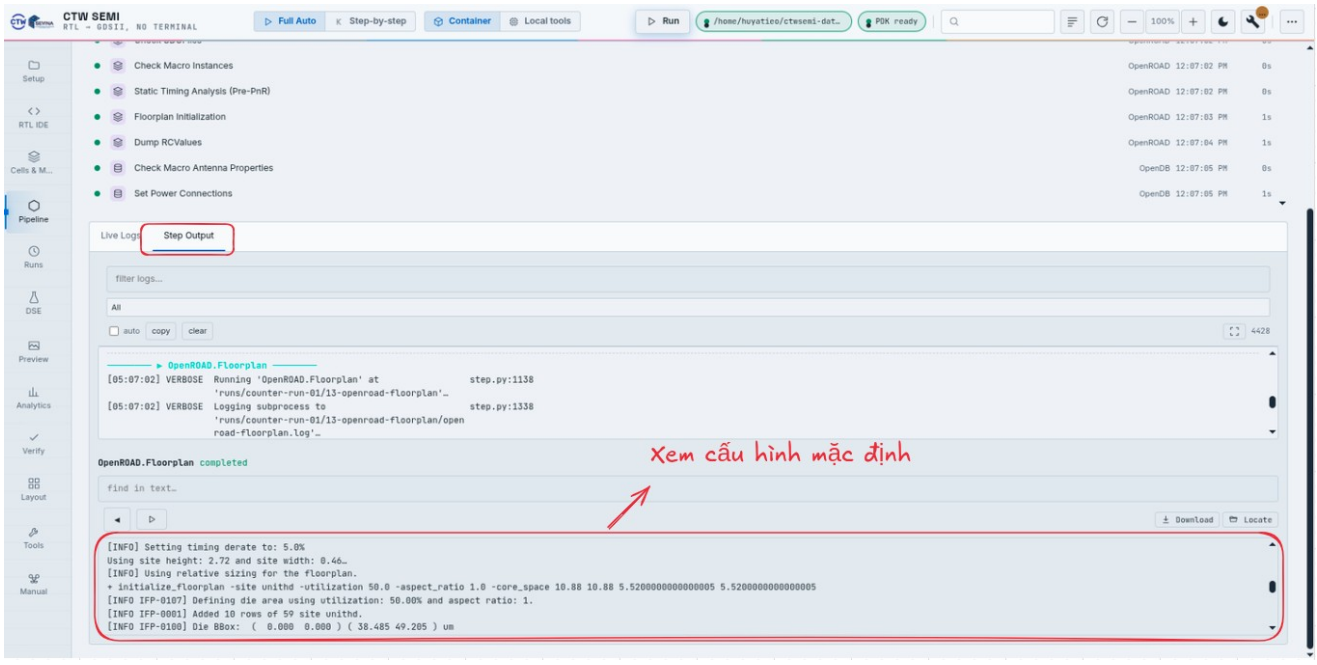
Cách đọc log: Nguyên tắc là đọc từ dưới ngược lên, Tìm dòng có từ khóa error, hoặc failed. Tìm được các dòng báo lỗi giống hình dưới.



Từ 2 dòng log khoanh ở trên, ta có thể chốt lại với lỗi: Detailed placement failed (DPL-0036). Đây là một lỗi vô cùng phổ biến. Nguyên nhân cốt lõi là **không gian bố trí chip quá chật chội**, khiến công cụ không thể tìm được chỗ trống hợp lệ để đặt hoặc các bộ đệm (Buffers) thêm vào trong quá trình tối ưu (log báo: Inserted 9 hold buffers). Cách nhanh nhất để sửa là **tăng diện tích core khi floorplan (quy hoạch diện tích)**. Xem chi tiết hơn về cách sửa lỗi tại: https://github.com/The-OpenROAD-Project/OpenLane/issues/1746

Cách sửa như sau:

B1: Kiểm tra trạng thái floorplan mặc định. Chọn **Floorplan Initialization.** Copy log trả về để phân tích**:**



→ Cấu hình hiện tại đang là:

```
initialize_floorplan
    -site unithd
    -utilization 50.0
    -aspect_ratio 1.0
    -core_space 10.88 10.88 5.52 5.52
```

- Giải thích: utilization 50.0 nghĩa là vùng diện tích của các standard cell sẽ chiếm 50% so với diện tích core, aspect_ratio 1.0, chiều và chiều rộng có tỉ lệ bằng nhau, yêu cầu core có hình vuông, core_space 10.88 10.88 5.52 5.52 là khoảng cách giữa core và biên die chip.

Bottom ≈ 10.88 µm

Top ≈ 10.88 µm

Left ≈ 5.52 µm

Right ≈ 5.52 µm

→ Diện tích của standard cell (Total instances are) và diện tích core (Core are) sau khi synthesis (tổng hợp thành các cổng logic standard cell).

```
[INFO IFP-0107] Defining die area using utilization: 50.00% and aspect ratio: 1.

[INFO IFP-0001] Added 10 rows of 59 site unithd.
[INFO IFP-0100] Die BBox:  (  0.000  0.000 ) ( 38.485 49.205 ) um
[INFO IFP-0101] Core BBox: (  5.520 10.880 ) ( 32.660 38.080 ) um
[INFO IFP-0102] Core area:                 738.208 um^2
[INFO IFP-0103] Total instances area:      376.611 um^2
[INFO IFP-0104] Effective utilization:     0.510
[INFO IFP-0105] Number of instances:       31

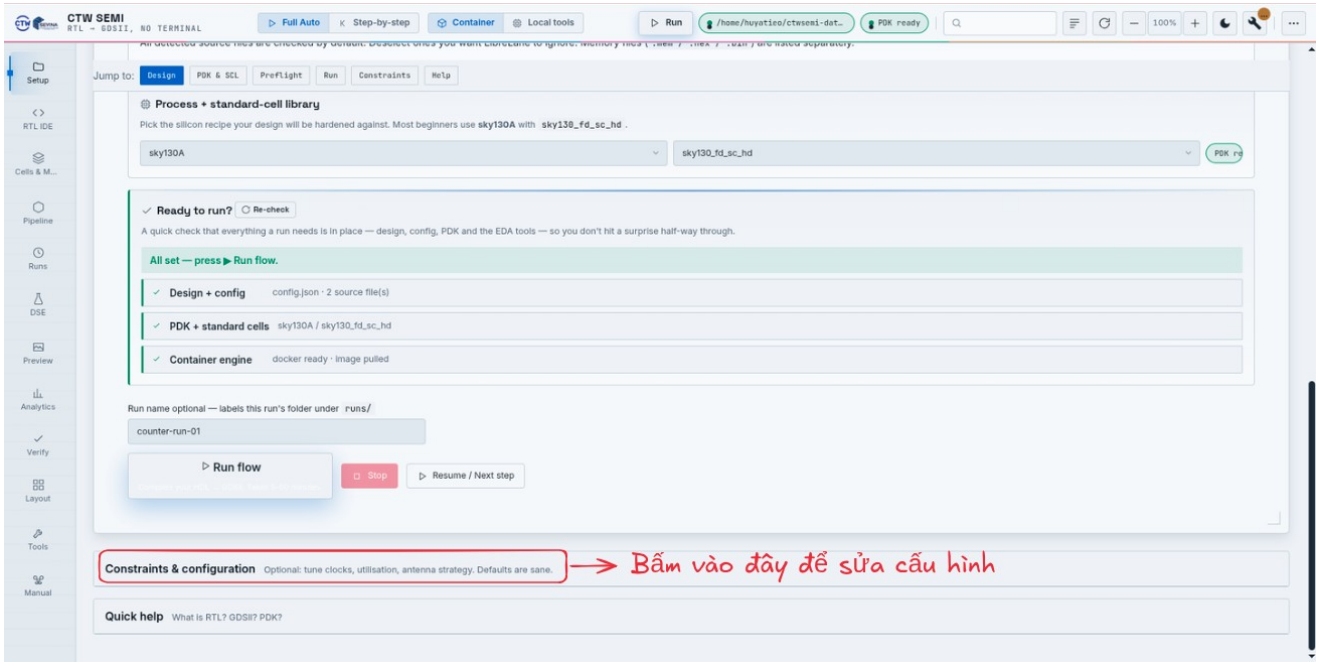
Cell type report:                          Count       Area

   Sequential cell                 8     170.16
   Multi-Input combinational cell  23    206.45
   Total                           31    376.61
```

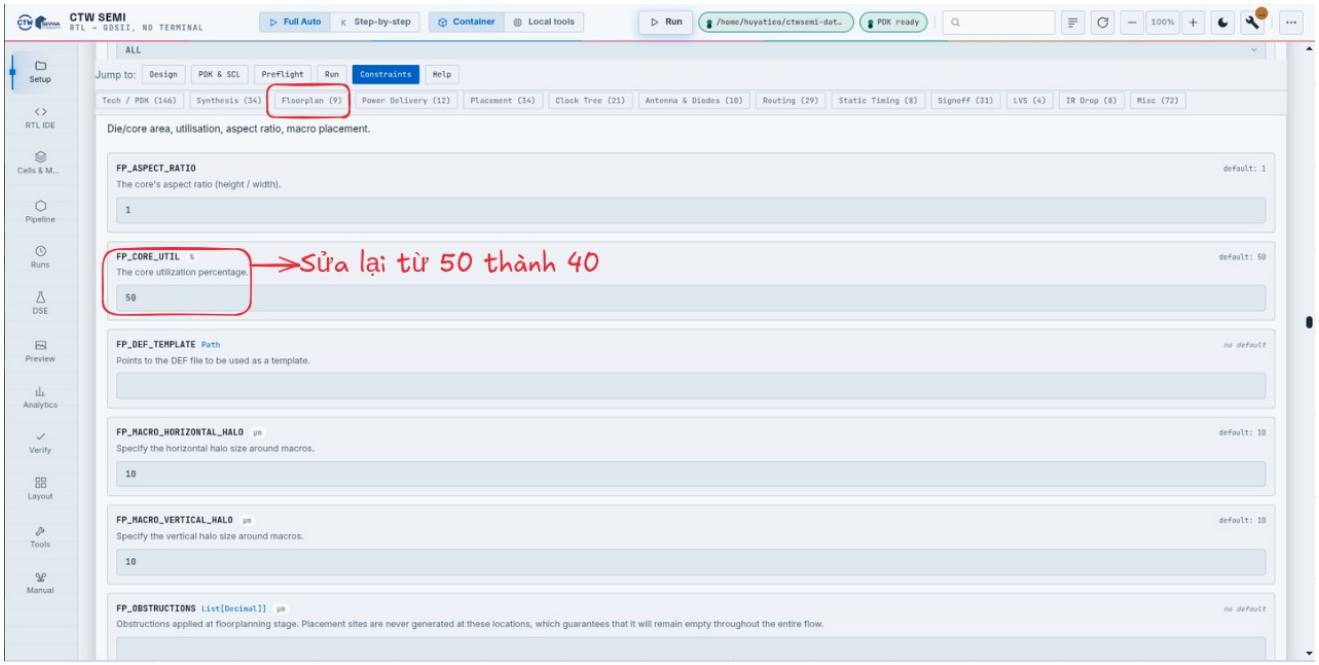
- Giải thích: Diện tích standard cell 376.611 um^2, Ta có công thức: Uitilization = Std cell area / Core area. Suy ra với Utilization 50%, ta tính được Core area: 738.208 um^2.

Lưu ý diện tích của core ban đầu chưa được cung cấp. Chỉ khi có diện tích standard cell, thì dựa vào utilization, aspect ratio để quy hoạch core.

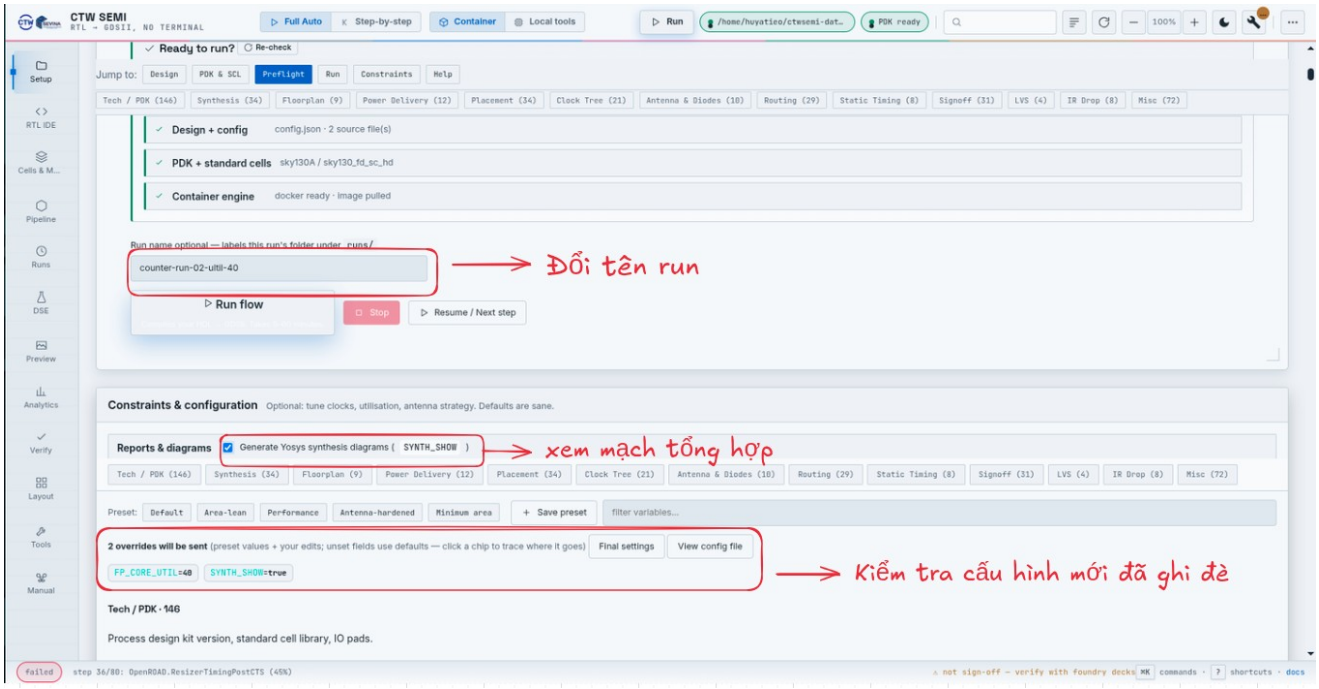
B1: Quay lại cửa sổ Setup. Chọn như hình dưới:



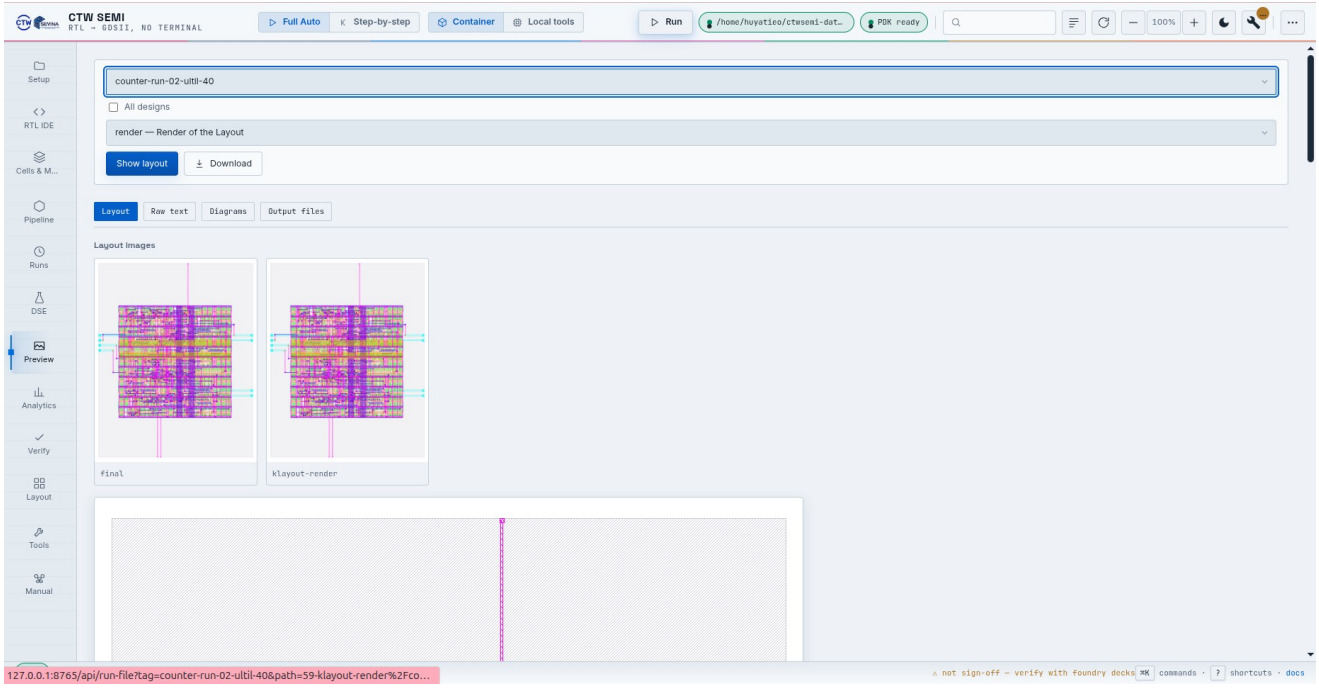
B2: Tăng diện tích core bằng cách sửa như sau:



B3: Chạy lại run mới với cấu hình mới đã ghi đè



Kết quả sau khi chạy lại run mới



Bài tập Lab 0

- Giải thích hoạt động của counter 8 bit với waveform.
- Vẽ sơ đồ phần cứng RTL của counter 8 bit.
- Hiểu cách khởi tạo floor plan.
- Hiểu về FP_CORE_UTIL.
- Tính lại instances area, core area với FP_CORE_UTIL = 45%.
- Đọc được log lỗi của step detailed placement routing (DPL-0036).